



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)
НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 Информатика и вычислительная техника**

О т ч е т

по лабораторной работе № 2

Название: **Изучение принципов работы микропроцессорного ядра RISC-V**
Дисциплина: **Архитектура ЭВМ**

Студент гр. ИУ7-54Б _____ А.А.Федин

(Подпись, дата)

(И.О. Фамилия)

Преподаватель _____ А.Ю. Попов

(Подпись, дата)

(И.О. Фамилия)

2025 год

ВВЕДЕНИЕ

Основной целью работы является ознакомление с принципами функционирования, построения и особенностями архитектуры суперскалярных конвейерных микропроцессоров. Дополнительной целью работы является знакомство с принципами проектирования и верификации сложных цифровых устройств с использованием языка описания аппаратуры SystemVerilog и ПЛИС.

1. Теоретическая часть

RISC-V является открытым современным набором команд, который может использоваться для построения как микроконтроллеров, так и высокопроизводительных микропроцессоров. В связи с такой широкой областью применения в систему команд введена вариативность. Таким образом, термин RISC-V фактически является названием для семейства различных систем команд, которые строятся вокруг базового набора команд, путем внесения в него различных расширений.

В данной работе исследуется набор команд RV32I, который включает в себя основные команды 32-битной целочисленной арифметики кроме умножения и деления. В рамках данного набора команд мы не будем рассматривать системные команды, связанные с таймерами, системными регистрами, управлением привилегиями, прерываниями и исключениями.

Набор команд RV32I предполагает использование **32 регистров общего назначения x0-x31 размером в 32 бита каждый** и регистр **pc**, хранящего адрес следующей команды. Все регистры общего назначения равноправны, в любой команде могут использоваться любые из регистров. Регистр pc не может использоваться в командах.

Архитектура RV32I предполагает плоское линейное 32-х битное адресное пространство. Минимальной адресуемой единицей информации является 1 байт. Используется порядок байтов от младшего к старшему (Little Endian), то есть, младший байт 32-х битного слова находится по младшему адресу (по смещению 0). Отсутствует разделение на адресные пространства команд, данных и ввода-вывода. Распределение областей памяти между различными устройствами (ОЗУ, ПЗУ, устройства ввода-вывода) определяется реализацией.

Большая часть команд RV32I является трехадресными, выполняющими операции над двумя заданными явно операндами, и сохраняющими результат в регистре. Операндами могут являться регистры или константы, явно заданные в коде команды. Операнды всех команд (кроме команды `auipc`) задаются явно. В том случае, если

операндами являются регистры, мы будем их называть **исходными регистрами (rs, source register)**, регистр, в который сохраняется результат — **целевым регистром (rd, destination register)**.

Теперь перейдем от рассмотрения абстрактной архитектуры системы команд к рассмотрению микроархитектуры ядра Taiga. Будем рассматривать систему, состоящую из вычислительного ядра Taiga и локальной памяти, реализованной с помощью блочной памяти ПЛИС. Данная память является статической, синхронной и двухпортовой. Один и тот же блок памяти используется для реализации как **памяти команд (ПК)**, так и **памяти данных (ПД)**. Таким образом команды и данные находятся в едином адресном пространстве. Дешифратор адресов настроен таким образом, что блок памяти ПЛИС отображается в адресное пространство RISC-V с адреса **0x80000000**, как мы это видели из рассмотрения примера выше.

Благодаря двухпортовой организации имеется возможность чтения и записи одновременно и команд и данных. Кроме того, блочная память ПЛИС имеет фиксированную задержку доступа в 1 такт. Таким образом, в нашей системе не будут возникать задержки доступа к памяти, в связи с чем отпадает необходимость в кеш-памяти.

Taiga является **конвейерным** микропроцессором с элементами суперскалярности. При конвейерной организации микропроцессора различные команды одновременно проходят различные стадии своей обработки. Конвейер Taiga насчитывает 4 стадии. В скобках приведены сокращенные обозначения стадий.

1. Выборка(F). Стадия, на которой команда извлекается из ПК. Выполняется в блоке выборки;
2. Диспетчеризация (ID). Стадия, на которой происходит запись команды в очередь команд для декодирования. Выполняется в блоке управления метаданными;
3. Декодирование и планирование на выполнение (D). Стадия на которой происходит определение типа и полей команды и определение вычислительного блока, способного ее исполнить. Выполняется в блоке декодирования и планирования на выполнение;

4. Выполнение (AL, M1..M3, в зависимости от исполнительного блока). Стадия, на которой команда передается в блок выполнения.

"Ширина" конвейера Taiga (то есть, количество команд, которые одновременно могут находиться на одной и той же стадии конвейера) равна 1 для всех стадий, кроме стадии выполнения. В лучшем случае, каждая стадия конвейера (кроме выполнения) выполняется за один такт.

В состав рассматриваемой конфигурации Taiga входит 3 блока выполнения команд: Арифметико-логическое устройство (АЛУ), блок доступа к памяти (LSU) и блок ветвлений. АЛУ и блок ветвлений выполняют команды за 1 такт, LSU — минимум за 3. Таким образом, возможна ситуация когда команда обращения к памяти выполняется одновременно с арифметической командой.

Ниже, на рисунке 1.1 приведена структурная схема ядра Taiga.

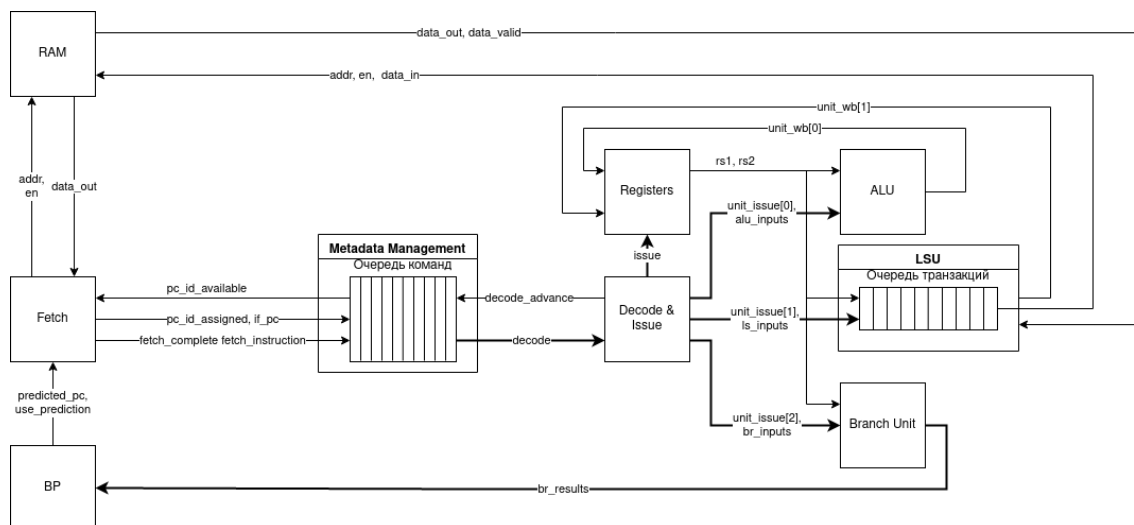


Рисунок 1.1 — Обобщенная структурная схема ядра Taiga

На рисунке показаны основные блоки, из которых состоит ядро Taiga: блоки, выполняющие стадии работы конвейера, исполнительные блоки, память команд и данных.

2. Практическая часть

Рассмотрим пример небольшой программы для RV32I, которым мы будем пользоваться далее для исследования процесса выполнения команд. Исходный текст исследуемой программы представлен на листинге 2.1

Листинг 2.1 — Листинг исходной программы

```
.section .text
.globl _start;
len = 8 #Размер массива
enroll = 4 #Количество обрабатываемых элементов за одну итерацию
elem_sz = 4 #Размер одного элемента массива

_start:
addi x20, x0, len/enroll
la x1, _x
loop:
lw x2, 0(x1)
add x31, x31, x2
lw x2, 4(x1)
add x31, x31, x2
lw x2, 8(x1)
add x31, x31, x2
lw x2, 12(x1)
add x31, x31, x2
addi x1, x1, elem_sz*enroll
addi x20, x20, -1
bne x20, x0, loop
addi x31, x31, 1
forever: j forever

.section .data
_x:
.4byte 0x1
.4byte 0x2
.4byte 0x3
.4byte 0x4
.4byte 0x5
.4byte 0x6
.4byte 0x7
.4byte 0x8
```

Дизассемблированный текст исследуемой программы представлен на листинге 2.2

Листинг 2.2 —Дизассемблированный листинг исходной программы

```
80000000 <_start>:
80000000:      00200a13      addi      x20,x0,2
80000004:      00000097      auipc     x1,0x0 (1)
80000008:      03c08093      addi      x1,x1,60 # 80000040 <_x>

8000000c <loop>:
8000000c:      0000a103      lw        x2,0(x1)
80000010:      002f8fb3      add       x31,x31,x2
80000014:      0040a103      lw        x2,4(x1)
80000018:      002f8fb3      add       x31,x31,x2
8000001c:      0080a103      lw        x2,8(x1)
80000020:      002f8fb3      add       x31,x31,x2
80000024:      00c0a103      lw        x2,12(x1)
80000028:      002f8fb3      add       x31,x31,x2
8000002c:      01008093      addi      x1,x1,16
```

```

80000030:      fffa0a13          addi    x20,x20,-1
80000034:      fc0a1ce3          bne     x20,x0,8000000c <loop>
80000038:      001f8f93          addi    x31,x31,1

8000003c <forever>:
8000003c:      0000006f          jal     x0,8000003c <forever>

```

Далее, согласно варианту № 20, необходимо продемонстрировать диаграммы для адреса 8000002с, на 2-й итерации, следовательно. на рисунке 2.1 приведена временная диаграмма, поясняющая этапы выборки и диспетчеризации.

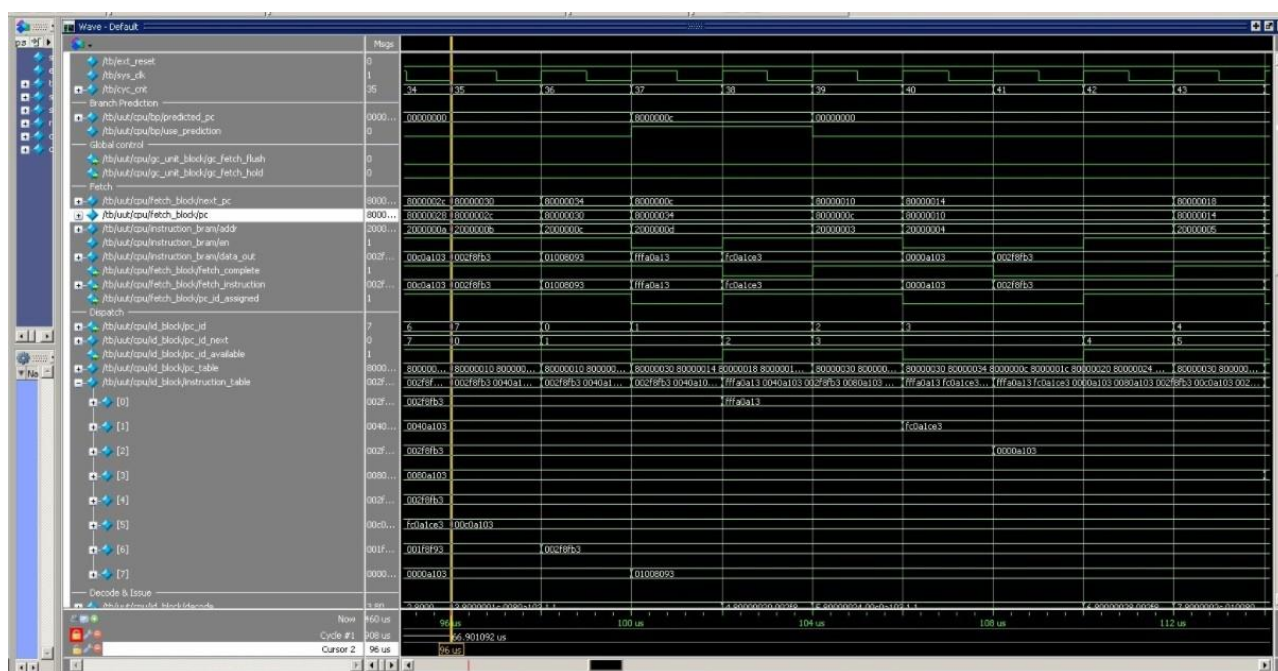


Рисунок 2.1 — Временная диаграмма выполнения стадий выборки и диспетчеризации команды

В соответствии с заданием 3 выполняем поиск такта, в котором выполняется декодирование и планирование команды с адресом 080000038, номер итерации: 2-я. На рисунке 2.2 приведена временная диаграмма, поясняющая этап декодирования и планирования.

Индивидуальный вариант:

На листинге 2.3 приведён индивидуальный вариант программы.

Листинг 2.3 — Листинг индивидуального варианта программы

```
.section .text
.globl _start;
len = 9 #Размер массива
enroll = 2 #Количество обрабатываемых элементов за одну итерацию
elem_sz = 4 #Размер одного элемента массива

_start:
    la x1, _x
    addi x20, x0, (len-1)/enroll
    lw x31, 0(x1)
    addi x1, x1, elem_sz*1
lp:
    lw x2, 0(x1)
    lw x3, 4(x1)
    add x1, x1, elem_sz*enroll
    bltu x2, x31, lt1
    add x31, x0, x2 #!
lt1:
    bltu x3, x31, lt2
    add x31, x0, x3
lt2:
    addi x20, x20, -1
    bne x20, x0, lp
lp2: j lp2

.section .data
_x:
    .4byte 0x1
    .4byte 0x2
    .4byte 0x3
    .4byte 0x4
    .4byte 0x8
    .4byte 0x6
    .4byte 0x7
    .4byte 0x5
    .4byte 0
```

На рисунках 2.4-2.7 изображены временные диаграммы сигналов, соответствующих всем стадиям выполнения команды, обозначенной в тексте программы вариантом 20 символом #!.

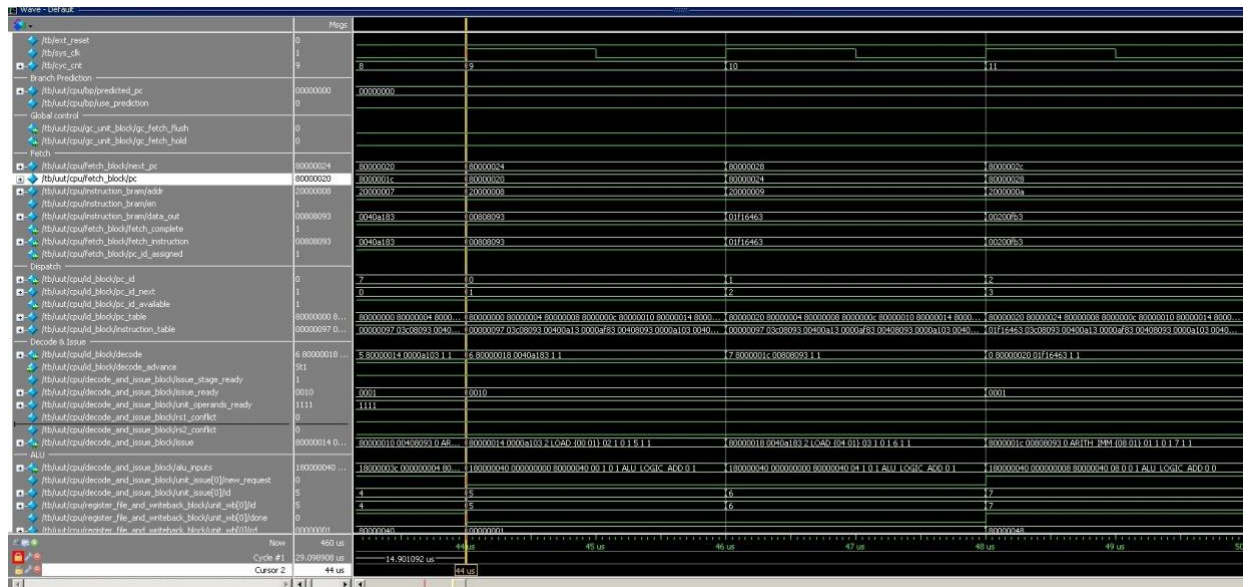


Рисунок 2.4 — Диаграмма, соответствующая этапу выборки команды, обозначенной в тексте программы символом #!

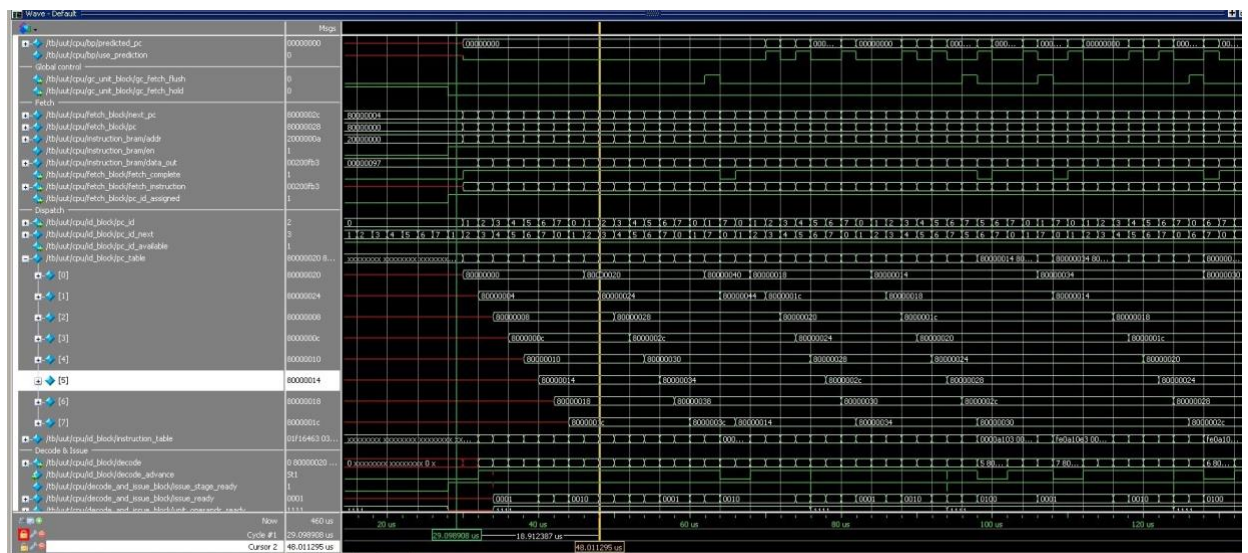


Рисунок 2.5 — Диаграмма, соответствующая этапу диспетчеризации команды

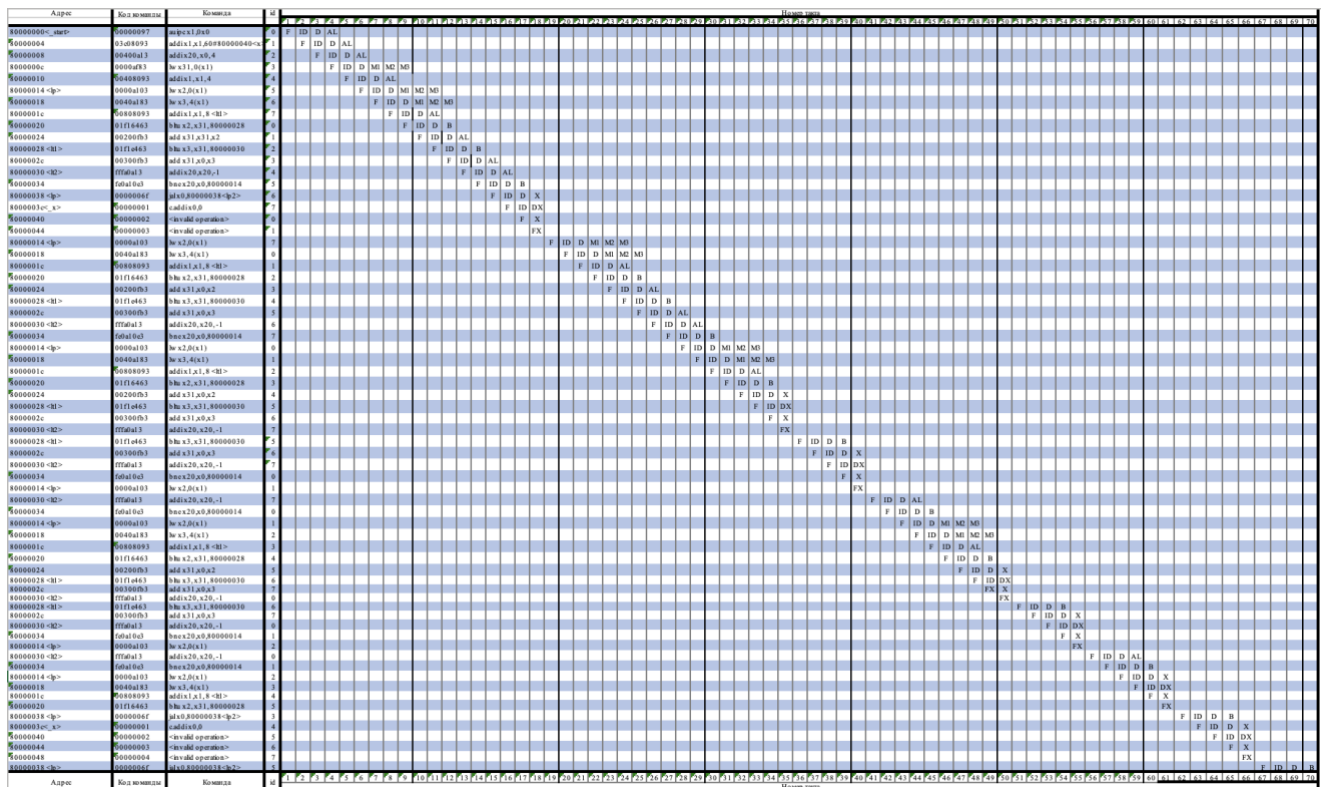


Рисунок 2.8 — Трасса работы программы

Из полученной выше трассы видно, что выполнение программы до окончания последней команды заняло 67 тактов. За время работы программы не случилось ни одного конфликта, однако случилось 7 gc_fetch_flush. Было принято решение убрать цикл и производить поиск максимального элемента без него. На листингах 2.4-2.5, представлены код и деассемблированный листинг оптимизированной программы, а на рисунка 2.9 - новая трасса программы.

Листинг 2.4 — Листинг оптимизированного варианта программы

```

.section .text
.globl _start;
len = 9 #Размер массива
enroll = 2 #Количество обрабатываемых элементов за одну итерацию
elem_sz = 4 #Размер одного элемента массива

_start:
    la x1, _x
    lw x31, 0(x1)
    addi x1, x1, elem_sz*1

    lw x2, 0(x1)
    lw x3, 4(x1)

```

1p2 :

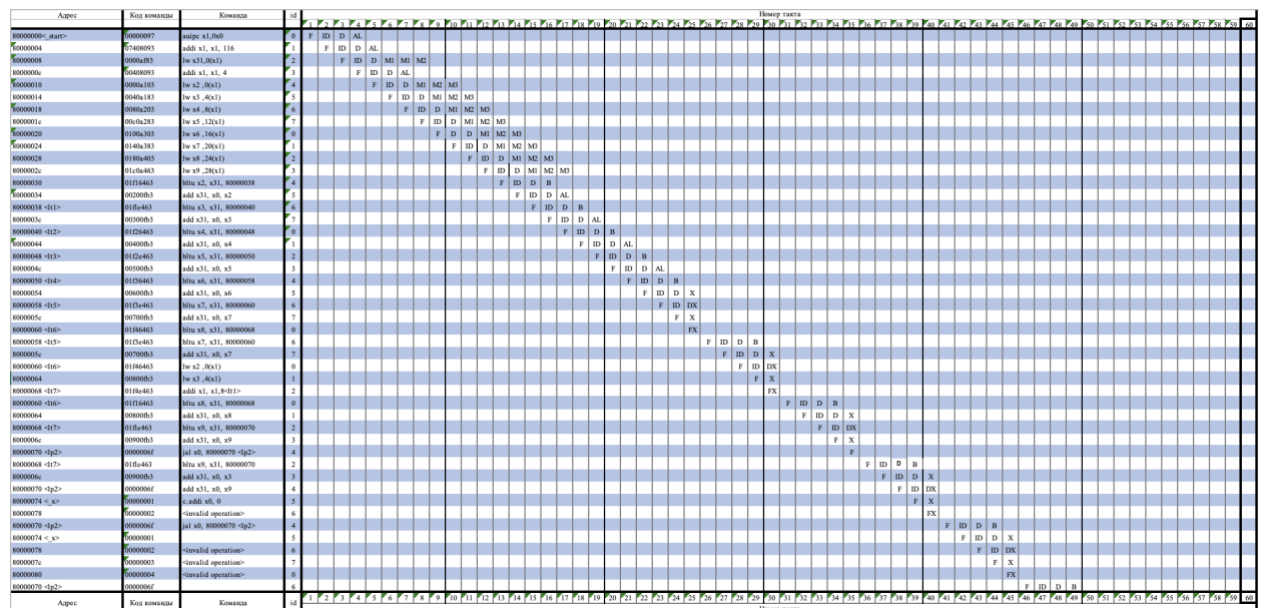


Рисунок 2.9 — Трасса работы оптимизированной версии программы

ВЫВОД

В данной лабораторной работе было проведено ознакомление с архитектурой ядра Taiga, а именно с порядком работы вычислительного конвейера: изучены команды RV32I, рассмотрены действия, выполняемые на каждой стадии конвейера, и данные, передаваемые между ними. После ознакомления с теоретической стороной вопроса, был выполнен разбор этапов выполнения программы на симуляции процессора с набором инструкций RV32I. После ее анализа удалось провести оптимизацию. В итоге, теоретические знания о порядке исполнения программ на процессорах с RISC архитектурой были закреплены на практике. Таким образом все поставленные задачи решены, основная цель работы достигнута.